R Lab: Foundations of Regression

Robert Gulotty

1/26/2017

```
# install.packages('plotrix')
library(plotrix)
```

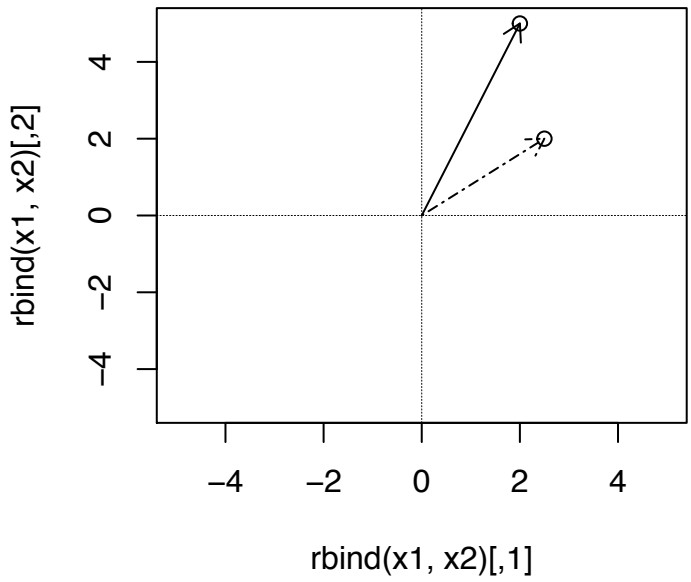
In this note, I describe the way that vectors and matrix concepts relate to linear regression. I assume you know about linear regression a little bit, that it produces a line which guesses the value of some dependent variable given the observation of an independent variable, that the amount that the line misses the data is called the residuals. You may even know that the way you calculate the slope coefficient in the bivariate case is with $\beta = \frac{cov(x,y)}{var(x)}$. Here I connect these ideas to vectors.

Observe the following two vectors, `x1` and `x2`. We will treat `x1` as the observation of the dependent variable, and `x2` as the observation of the independent variable, like columns in a regular data set. The first observation has an independent variable outcome of 2.5 and an outcome of 2, the second observation has an independent variable outcome of 2 and an outcome of 5.

```
x1 <- c(2, 5)
x2 <- c(2.5, 2)
```

It is sometimes easier to visualize vectors in $\mathbb{R}^2$. Here we plot the two vectors, where x_1 is solid, and x_2 is dashed.

```
plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5))
abline(h = 0, lty = 2, lwd = 0.2)
abline(v = 0, lty = 2, lwd = 0.2)
arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1, lty = 6)
```



In R, when we use %*% on vectors, we get the dot product. The dot product is the sum of the products of two equal lenght vectors. The square root of a dot product of a vector with itself is the **Euclidean Length** of the vector, denoted $||\boldsymbol{x}||$. One useful formula is $\frac{\boldsymbol{x_1} \cdot \boldsymbol{x_2}}{||\boldsymbol{x}||}$, which is the linear algebra way of saying the correlation between $\boldsymbol{x_1}$ and $\boldsymbol{x_2}$ normalized by the standard deviation of $\boldsymbol{x_2}$. It can also be though of as the amount that vector $\boldsymbol{x_1}$ follows the direction of $\boldsymbol{x_2}$.

```
compx1x2 <- x1 %*% x2/sqrt(x2 %*% x2)
compx1x2
```

```
##          [,1]
## [1,] 4.685213
```

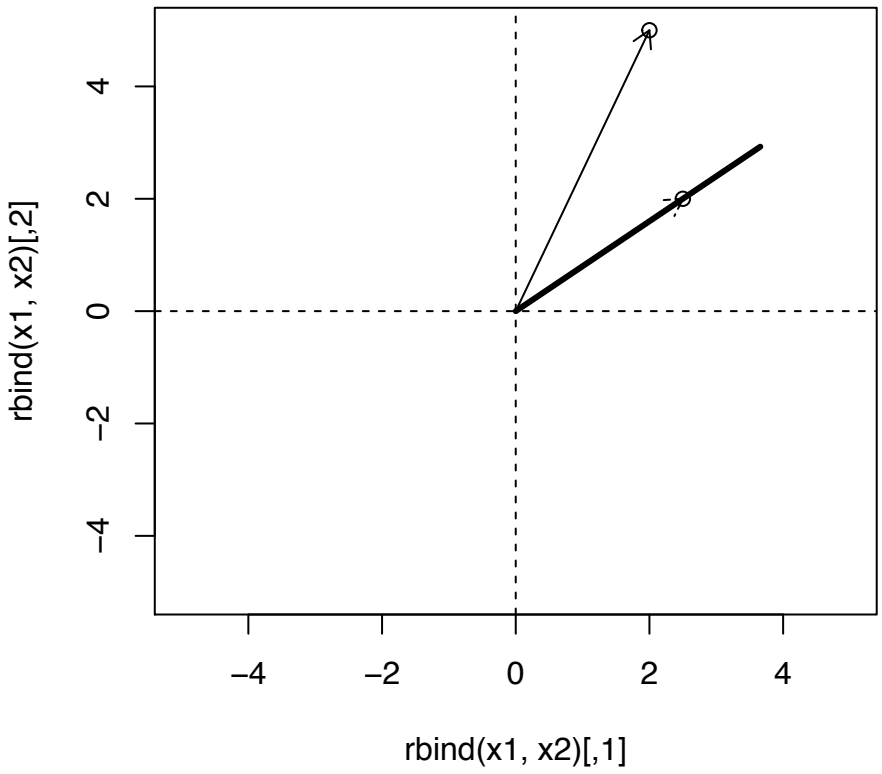
What makes this number special is that if we multiply it by $\boldsymbol{x_2}$ we get the **projection** of $\boldsymbol{x_1}$ onto $\boldsymbol{x_2}$. What I mean by projection is literally the shadow that $\boldsymbol{x_1}$ casts on $\boldsymbol{x_2}$

```
xbeta <- c((x1 %*% (x2)/x2 %*% x2)) * x2
xbeta
```

```
## [1] 3.658537 2.926829
```

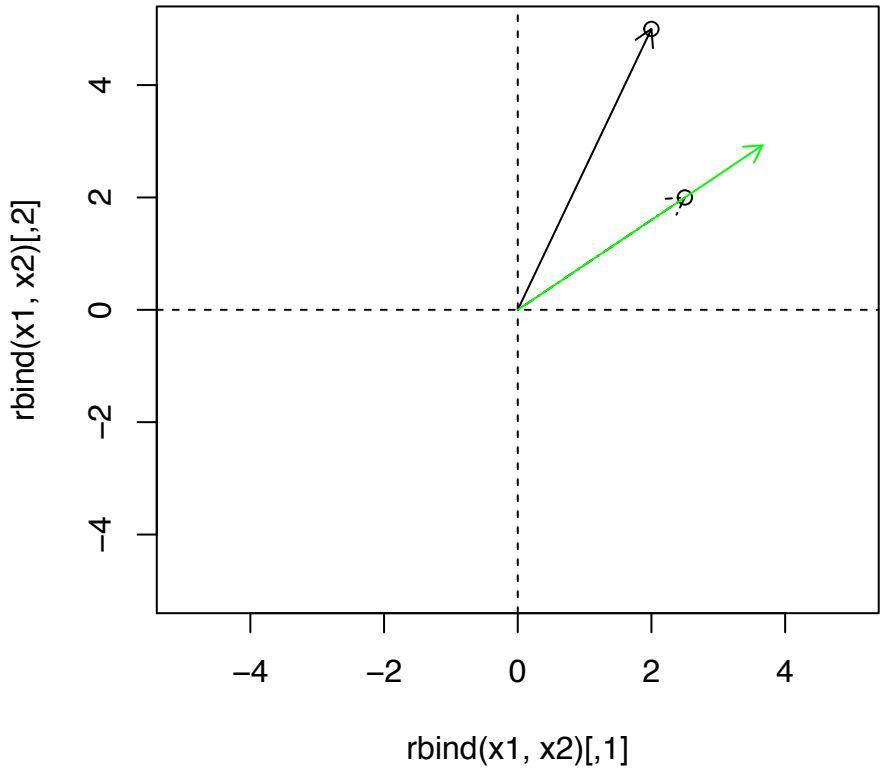
To show that this is in fact the shadow of x1 in the direction of x2, we will plot the component of $\boldsymbol{x_1}$ in the direction of $\boldsymbol{x_2}$. To do that, we will plot `compx1x2` along the same angle as $\boldsymbol{x_2}$, which we can recover using trigonometry. Here we use the fact that the arctangent or inverse tangengt of the ratio of the opposite side of a triangle to the adjacent side is the angle between them. We then can pass that angle into cosine to get the x length and to sine for the y length each multiplied by `compx1x2` to find the end points.

```
plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5))
abline(h = 0, lty = 2)
abline(v = 0, lty = 2)

arrows(x0 = 0, y0 = 0, x1 = x1[1], y1 = x1[2], length = 0.1)
arrows(x0 = 0, y0 = 0, x1 = x2[1], y1 = x2[2], length = 0.1,
    lty = 6)

arrows(x0 = 0, y0 = 0, x1 = cos(atan(x2[2]/x2[1])) * compx1x2,
    y1 = sin(atan(x2[2]/x2[1])) * compx1x2, lwd = 3, length = 0)
```


rbind(x1, x2)[,2]
4
2
0
−2
−4
−4
−2
0
2
4
rbind(x1, x2)[,1]

Now compare the prior result with the projection `xbeta` of x1 onto x2:

```
plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5))
abline(h = 0, lty = 2)
abline(v = 0, lty = 2)

arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1, lty = 6)
arrows(0, 0, xbeta[1], xbeta[2], length = 0.1, col = "green")
```



They are identical! What does this mean? Well consider the regression of $\boldsymbol{x_1}$ onto $\boldsymbol{x_2}$. (without an intercept).

```
ols1 <- lm(x1 ~ x2 - 1)
```

Compare `xbeta` (the projection) with the predictions from the linear model:

```
list(xbetaprojection = xbeta, OLSprediction = predict(ols1),
    OLSpredictionbyHand = ols1$coeff[1] * x2)
```

```
## $xbetaprojection
## [1] 3.658537 2.926829
##
## $OLSprediction
##        1        2
## 3.658537 2.926829
##
## $OLSpredictionbyHand
## [1] 3.658537 2.926829
```

All three are the same!

The fundamental reason for this is that regression **projects** your dependent variable onto the vector space spanned by your independent variable(s).

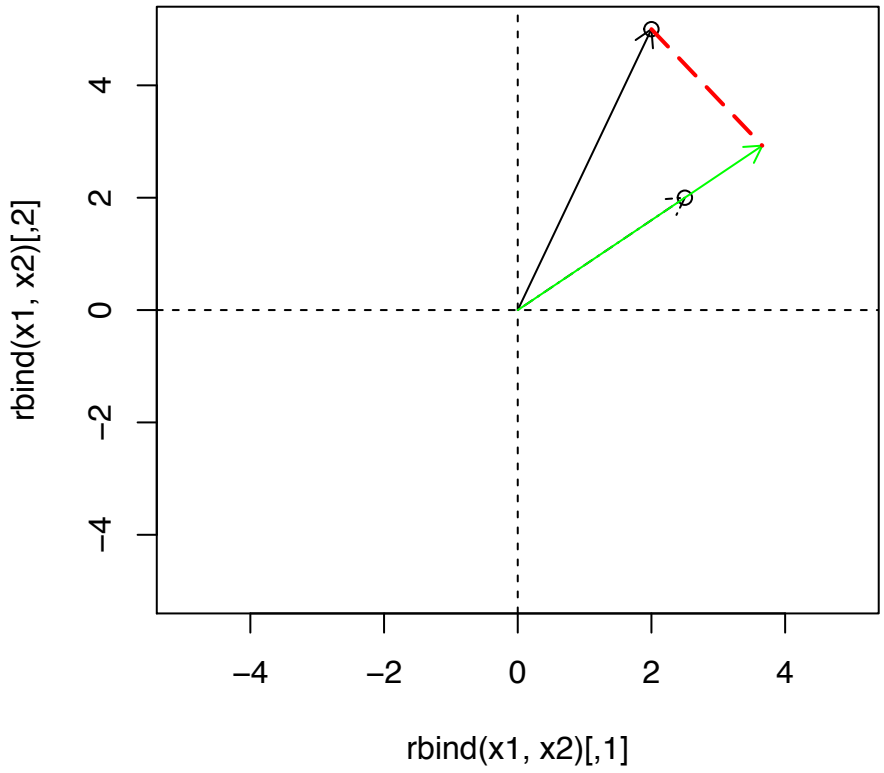
Note we can recover the length of the OLS prediction as well, and you will see that it is the same as before.

```
ols1$coeff[1] * sqrt(x2 %*% x2)
```

```
##          [,1]
## [1,] 4.685213
```

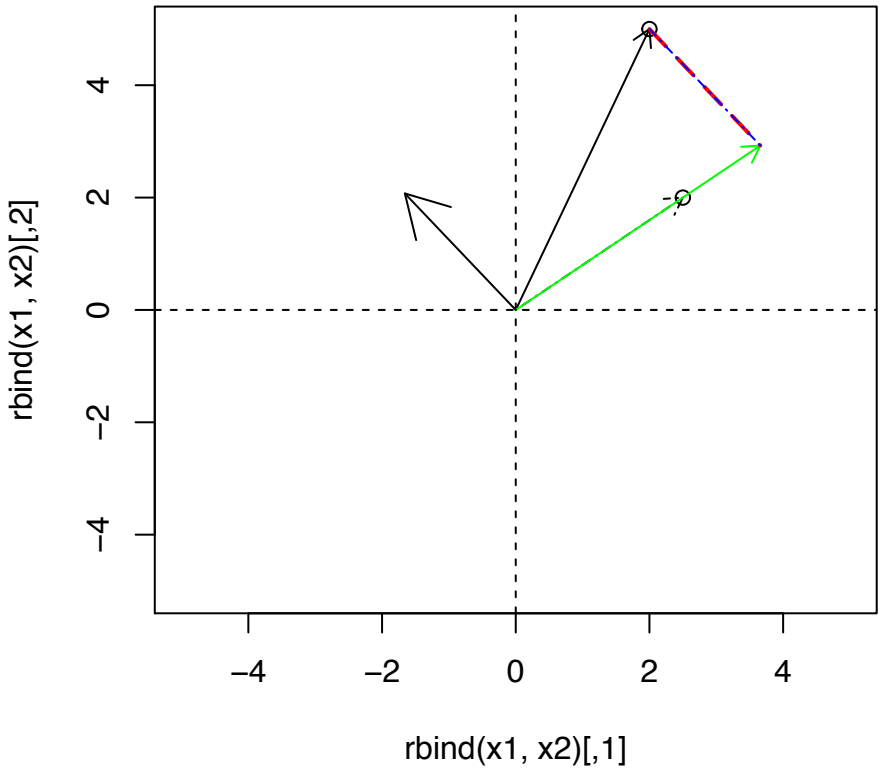
We can add a line that connects $\boldsymbol{x_1}$ and the point closest to $\boldsymbol{x_1}$ in the line spanned by $\boldsymbol{x_2}$

```
theta <- atan(x2[2]/x2[1])
plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5))
abline(h = 0, lty = 2)
abline(v = 0, lty = 2)

arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1, lty = 6)
arrows(0, 0, xbeta[1], xbeta[2], length = 0.1, col = "green")
arrows(x1[1], x1[2], cos(theta) * compx1x2, sin(theta) * compx1x2,
    length = 0, lty = 5, lwd = 2, col = "red")
```



That line is identical to the vector produced by the **residuals**, shifted to start at the projection.

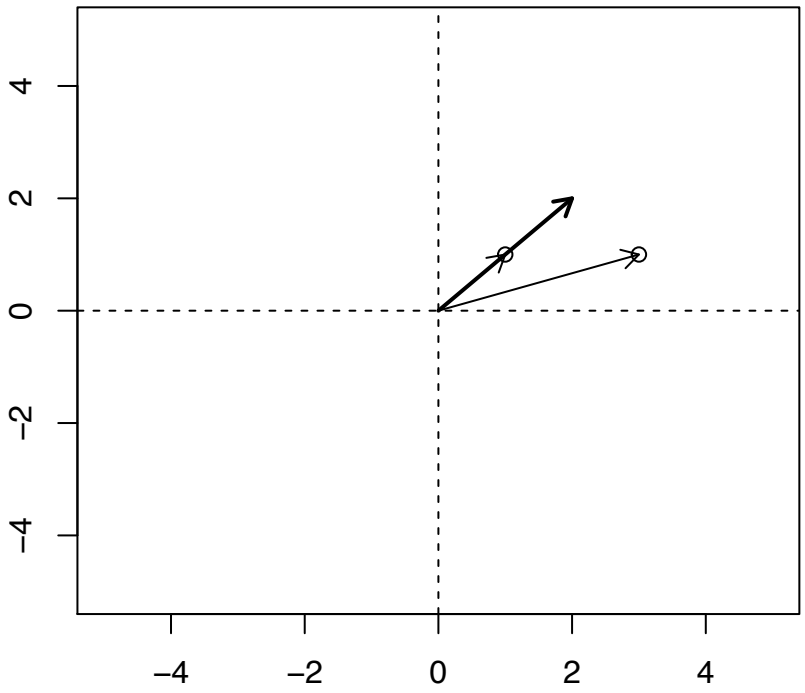
```
theta <- atan(x2[2]/x2[1])

plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5))

abline(h = 0, lty = 2)
abline(v = 0, lty = 2)

arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1, lty = 6)
arrows(0, 0, xbeta[1], xbeta[2], length = 0.1, col = "green")
arrows(x1[1], x1[2], cos(theta) * compx1x2, sin(theta) * compx1x2,
    length = 0, lty = 5, lwd = 2, col = "red")

arrows(0, 0, resid(ols1)[1], resid(ols1)[2])
arrows(ols1$coeff[1] * x2[1], ols1$coeff[1] * x2[2], resid(ols1)[1] +
    xbeta[1], resid(ols1)[2] + xbeta[2], length = 0, lty = 6,
    col = "blue")
```



Example 2: projection onto a vector of ones.

Now suppose rather than projecting $\boldsymbol{x_1}$ onto $\boldsymbol{x_2}$ from before, we just project it onto a vector of 1s.

```
x1 <- c(3, 1)
x2 <- c(1, 1)



plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5), xlab = "",
    ylab = "")
abline(h = 0, lty = 2)
abline(v = 0, lty = 2)
library(plotrix)
arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1)



ols1 <- lm(x1 ~ x2 - 1)


arrows(0, 0, ols1$coeff[1] * x2[1], ols1$coeff[1] * x2[2], length = 0.1,
    lwd = 2)
```



Each element of xbeta for this case is the mean of y:

```
c((x1 %*% (x2)/x2 %*% x2)) * x2
## [1] 2 2
ols1$coeff[1] * x2[1]
## x2
##  2
```

We can plot it just as before:

```
plot(rbind(x1, x2), xlim = c(-5, 5), ylim = c(-5, 5), xlab = "",
    ylab = "")
abline(h = 0, lty = 2)
abline(v = 0, lty = 2)
library(plotrix)
arrows(0, 0, x1[1], x1[2], length = 0.1)
arrows(0, 0, x2[1], x2[2], length = 0.1)



ols1 <- lm(x1 ~ x2 - 1)


arrows(0, 0, ols1$coeff[1] * x2[1], ols1$coeff[1] * x2[2], length = 0.1,
    lwd = 2)

arrows(ols1$coeff[1] * x2[1], ols1$coeff[1] * x2[2], resid(ols1)[1] +
    ols1$coeff[1], resid(ols1)[2] + ols1$coeff[1], length = 0.1,
    lty = 6, col = "blue")
```